



Agentic AI Bootcamp



Learn to Build AI That Thinks, Plans, and Acts.



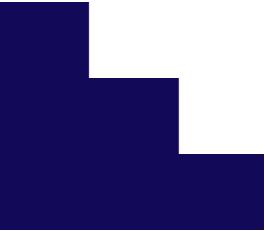
Agent Observability and Monitoring

Prepared by WeCloudData





Agenda



01 Importance of Observability in Agent Systems

02 Execution Traces

03 Monitoring System Behavior

04 Logging and Debugging Tools

05 Cost and Latency Tracking

Importance of Observability in Agent Systems



Why Observability is Critical for AI Agents

Agent Observability Matters

- AI agents are **non-deterministic systems**
- Same prompt can produce:
 - Different reasoning paths
 - Different tool usage
 - Different outputs
- Traditional logs are not enough
- Need visibility into:
 - Reasoning steps
 - Tool execution
 - Retrieval quality
 - Decision flow

If you cannot see how an agent thinks, you cannot reliably debug or improve it.





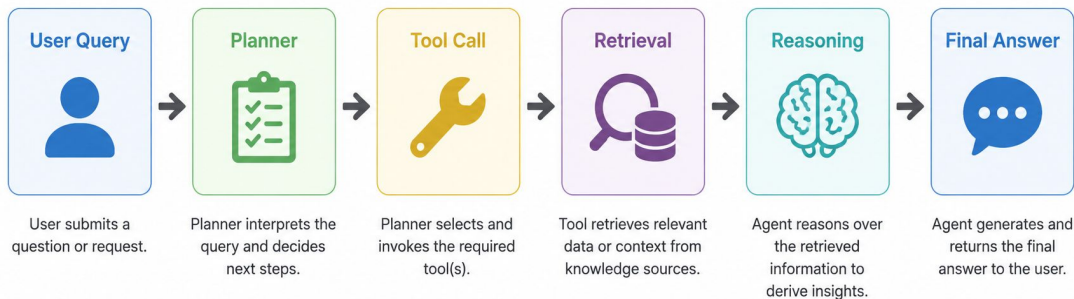
Defining Agent Observability

Agent Observability Matters

Agent Observability = Complete Visibility Into Agent Execution

Includes monitoring of:

- User input
- Prompt construction
- Memory usage
- Retrieval results (RAG)
- Tool calls
- Reasoning chains
- Intermediate decisions
- Final response



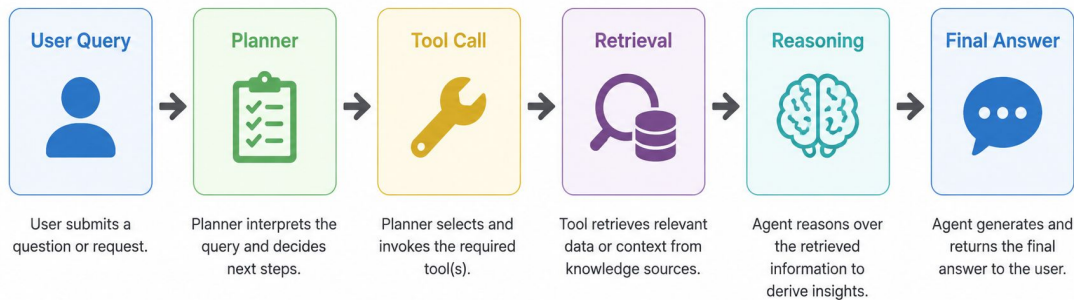


Defining Agent Observability

Agent Observability Matters

Observability helps answer:

- What happened?
- Why did it happen?
- Where did it fail?





The Non-Determinism Challenge

Why Traditional Debugging Fails for Agents

1. Traditional Software (Deterministic Flow)



2. AI Agents (Non-Deterministic Flow)





The Non-Determinism Challenge

Why Traditional Debugging Fails for Agents

Problems:

- Same query may behave differently each time
- Tool selection may vary
- Retrieval results may change
- LLM reasoning is probabilistic

Guessing the root cause becomes impossible without traces.



Finding Failure Points

Observability Helps Identify Hidden Failures

Common Agent Failure Types:

- Irrelevant document retrieval
- Wrong tool selection
- Hallucinated tool parameters
- Infinite reasoning loops
- Memory corruption
- Prompt injection effects
- Multi-agent coordination failures

Observability Enables:

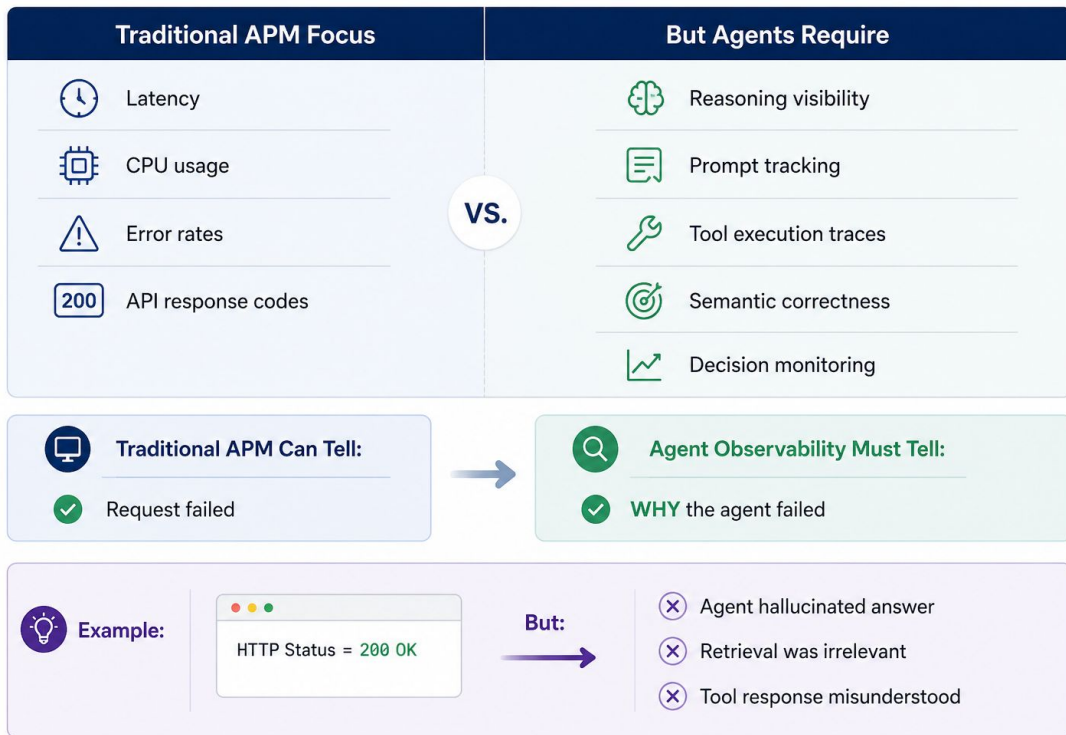
- Root cause analysis
- Faster debugging
- Better prompt refinement
- Safer deployments





The Gap in Traditional APM

Why Traditional Monitoring is Not Enough





From Prototyping to Production

Observability Maturity Model

Stage	Monitoring Style	Problem
Prototype	Print statements	Works only for small demos
Development	Manual trace review	Time-consuming
Pre-Production	Basic dashboards	Limited scalability
Production Scale	Full observability platform	Required for reliability





From Prototyping to Production

Observability Maturity Model

Production Reality:

- Thousands of agent runs daily
- Human review becomes impossible
- Automated monitoring becomes essential

Scale breaks manual debugging.



Observability for Governance & Compliance

SLAs, Compliance, and Trust

Organizations Need Proof That:

- Agents followed policy
- Sensitive data was handled correctly
- Approved tools were used
- Unsafe actions were avoided

Observability Provides:

- Full execution traces
- Auditability
- Compliance verification
- Incident investigation support

Important for:

- Healthcare
- Finance
- Legal systems
- Enterprise copilots

Traces become the “black box recorder” for AI systems.



When Agents “Pass” but Still Fail

The Semantic Success Trap

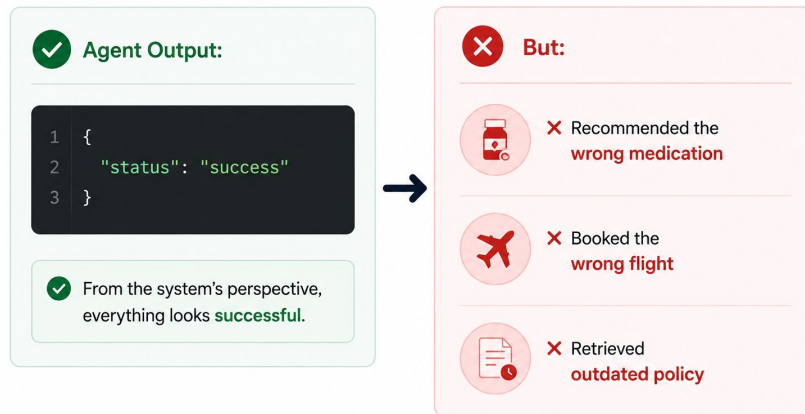
Syntactic Success:

- Valid JSON
- Correct API format
- HTTP 200 response
- No system errors

Semantic Failure:

- Wrong answer
- Misunderstood user intent
- Incorrect reasoning
- Unsafe recommendation
- Fake confidence

Example:



A successful response is not always a correct response.

Execution Traces



Understanding Execution Traces

What is an Execution Trace?

Execution Trace: A chronological record of everything an agent does during execution.

A Trace Captures:

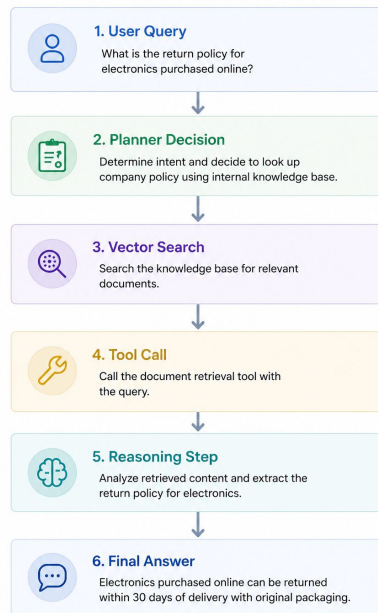
- User input
- Prompt construction
- Reasoning steps
- Tool invocations
- Retrieval outputs
- Memory interactions
- Final response
- Latency and token costs

Why It Matters:

Execution traces allow teams to:

- Reproduce failures
- Debug reasoning
- Analyze performance
- Improve reliability

Example Trace:





Hierarchical Traces vs Traditional Logs

From Flat Logs to Execution Trees



Traditional Logs:



Request Received



Tool Called



Response Returned



Problems:



Missing relationships



No reasoning context



Hard to reconstruct flow

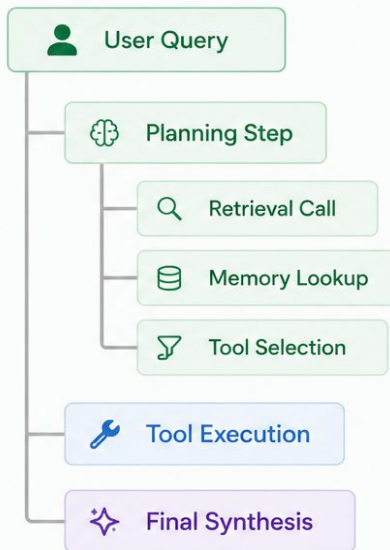


Modern Agent Traces:



Represent execution as a **tree structure**

Example Hierarchy:



Benefits:



Shows
dependency
chains



Tracks
nested
decisions



Makes
debugging
easier





Core Instrumentation Categories

Instrumentation: What Must Be Captured?

1. LLM Interactions

- Prompts
- System instructions
- Completions
- Token usage
- Model version

2. Reasoning Transitions

- Chain-of-thought transitions
- Planner decisions
- Goal decomposition

3. State Changes

- Agent state updates
- Session variables
- Task progress

4. Memory Operations

- Memory reads
- Memory writes
- Context injections

5. Retrieval & Tooling

- Vector search queries
- Retrieved documents
- Tool parameters
- Tool responses

If it influences the agent's behavior, it should be traceable.





Why High-Fidelity Traces Matter

High-Fidelity Tracing Principles

High-Fidelity = No Information Loss

Goals:

- Reproduce execution exactly
- Enable reliable regression testing
- Support training and evaluation
- Diagnose subtle failures

What Low-Fidelity Logging Misses:

- Hidden prompts
- Intermediate reasoning
- Failed retrievals
- Rejected tool calls
- State mutations

High-Fidelity Trace Includes:

- Every model invocation
- Every intermediate output
- Every external interaction

If it influences the agent's behavior, it should be traceable.



When Agents “Pass” but Still Fail

The Semantic Success Trap

1. Cognitive Surface

Internal thinking behavior of the agent

Includes:

- Raw prompts
- Intermediate completions
- Planner thoughts
- `<thinking>` segments
- Self-reflection loops

Example:

Need customer order history before refund decision. Searching database...

2. Contextual Surface

Everything happening outside the model

Includes:

- API calls
- Database queries
- Vector store retrievals
- Web searches
- Tool outputs

Key Idea: Agents fail because of both:

Bad reasoning

Bad external context



Threads, Sessions, and Multi-Turn Analysis

Why Single Traces Are Not Enough

Without Session Tracking:

- Context gets fragmented
- Failures appear isolated
- Goal completion cannot be measured

Need:

- Thread IDs
- Session-level traces
- Conversation timelines

Multi-Turn Failure Modes:

- Context drift
- Forgotten constraints
- Contradictory memory
- Goal divergence over time



Problem:

Users interact with agents across multiple turns.

Example:



Turn 1:



"Book me a hotel."



Turn 2:



"Actually make it near the airport."



Turn 3:



"Use my company discount."



Monitoring System Behavior



System-Level Monitoring for AI Agents

Monitoring Beyond the Application Layer

Traditional monitoring focus on Application logs, API responses, Service metrics

Problem:

Agent application code changes rapidly:

- Prompt updates
- New tools
- New workflows
- Multi-agent orchestration

Solution: Boundary Tracing

Monitor stable system interfaces instead:

- Kernel events
- Network activity
- File operations
- Process behavior

Why Boundary Monitoring Matters:

- More reliable visibility
- Independent of app implementation
- Captures actual system effects

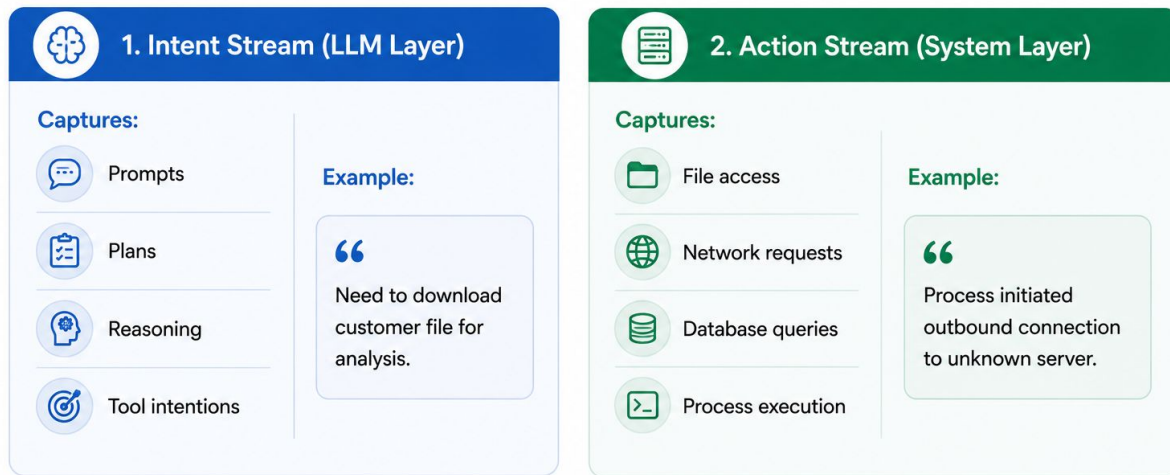




Monitoring What the Agent Thinks vs What It Does

Intent vs Action Stream

Two Critical Streams



Why Both Matter:

An agent may appear safe in reasoning, but **dangerous actions** may occur at the system level.



Intent Stream
(LLM Layer)



Action Stream
(System Layer)





Detecting Security Risks & Failures

Detecting Prompt Injections and Dangerous Behaviors

Observability Captures the Attack Chain:

1. Agent fetches malicious URL
2. Prompt becomes compromised
3. Agent attempts unauthorized action
4. Suspicious outbound traffic detected

AI Watching AI

Secondary observer models can:

- Analyze traces
- Detect anomalies
- Flag suspicious patterns
- Identify risky behavior

Other Detectable Problems:

- Unauthorized API usage
- Data exfiltration
- Infinite reasoning loops
- Excessive retries
- Resource spikes
- Tool misuse



Continuous Monitoring in Production

Production Monitoring & Its Limitations

Modern AI Systems Require:

- Continuous statistical monitoring instead of:
 - One-time testing
 - Static release approval

Production Monitoring Tracks:

- Failure rates
- Latency spikes
- Hallucination frequency
- Tool failure patterns
- Resource consumption
- Behavioral drift over time





Continuous Monitoring in Production

Production Monitoring & Its Limitations

Important Limitation:

Monitoring alone is NOT enough.

Why?

Dashboards detect problems *after deployment*.

Also Needed:

- Pre-deployment testing
- Safety evaluations
- Red-team testing
- Formal verification methods

Observability improves visibility, but reliability requires both monitoring AND rigorous testing.



Logging and Debugging Tools



Observability Frameworks & Standards

Modern Observability Frameworks for AI Agents

OpenTelemetry (OTel)

Emerging industry standard for:

- Vendor-neutral observability
- Distributed tracing
- Metrics and logging
- AI system instrumentation

Why OTel Matters:

- Standardized trace format
- Cross-platform compatibility
- Integrates with existing monitoring stacks
- Supports production-scale systems

AgentTrace Framework

Schema-based framework for:

- Capturing cognitive artifacts
- Tracking reasoning chains
- Linking operational signals with agent behavior

Combines:

- LLM reasoning traces
- Tool usage
- Memory operations
- System telemetry

Standardization enables interoperability across agent ecosystems.





Framework-Native Tools vs OTel-Based Systems

Comparing Observability Platforms

Framework-Native Tools vs OTel-Based Systems

Approach	Advantages	Limitations
 Framework-Native	 Easy setup, deep integration	 Vendor lock-in
 OTel SDKs	 Portable, standardized	 More complex setup



Framework-Native Tools vs OTel-Based Systems

Comparing Observability Platforms

Laminar (Best for)	Langfuse(Best for)	LangSmith (Best for)
Long-running autonomous agents	Prompt management	LangGraph debugging
Multi-step execution monitoring	Experiment tracking	Trace visualization
Production observability	Cost analysis	Workflow evaluation





Using AI to Debug AI Systems

AI-Assisted Debugging & Regression Testing

Benefits:

- Faster root cause analysis
- Easier trace exploration
- Reduced manual investigation
- More accessible debugging workflows

Stochastic Regression Testing

Traditional Software:

Pass / Fail

AI Systems Require:

- PASS
- FAIL
- INCONCLUSIVE

Why?

Agent outputs are probabilistic, not deterministic.

AI-Assisted Debugging

Natural language debugging assistants can answer questions like:

Debug Assistant
Ask questions about your AI agent's behavior and get instant insights.

- Understand failures
- Trace root causes
- Get actionable insights

Why did the model hallucinate in step 3? 10:21 AM

In step 3, the model generated an answer not grounded in the retrieved context. The prompt lacked sufficient constraints and the retrieved documents were semantically weak.

Which retrieval caused the failure? 10:22 AM

The failure was caused by Retrieval Call #2. It returned an irrelevant document (doc_id: 8f3a...) with low relevance score (0.21).

Why did latency spike during tool execution? 10:23 AM

Latency spiked during the "SearchFlights" tool execution because the external API had high response time (p95: 3.2s) due to elevated error rates and retries.





Continuous Evaluation & Behavioral Contracts

Production Testing and Runtime Enforcement

**CI/CD Pipelines
Now Include**

-  Statistical evaluation thresholds
-  Hallucination rate checks
-  Safety benchmarks
-  Tool reliability metrics

**AgentAssay**

Token-efficient regression testing framework that:

-  Reduces evaluation cost
-  Maintains statistical confidence
-  Minimizes repeated test runs



**Trace-First
Offline Analysis**

Reuse production traces to:

-  Detect regressions
-  Perform contract checking
-  Expand test coverage at low cost



**Agent Behavioral
Contracts (ABC)**

Formal rules defining:

-  Allowed behaviors
-  Safety constraints
-  Compliance requirements

Example:
 Agent must never expose customer PII externally.

Modern agent observability combines tracing, statistical testing, and runtime enforcement into a unified reliability pipeline.



Cost and Latency Tracking



Why Agentic Workflows Become Expensive

Understanding Token Economics

Cost Attribution in Agent Systems

Observability helps identify:

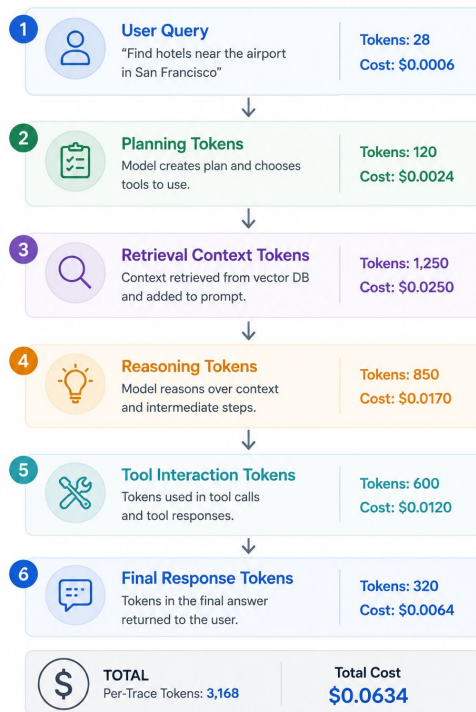
- Which tasks consume the most tokens
- Which tools trigger expensive calls
- Which reasoning chains are inefficient

Coding Agents 100x–1000x more than normal chat

Reasons:

- Large codebase ingestion
- Repeated context loading
- Multi-step reasoning
- Long-horizon planning
- Iterative debugging loops

Per-Trace Cost Breakdown:





Managing Unpredictable Token Usage

Consumption Variability & Budget Control

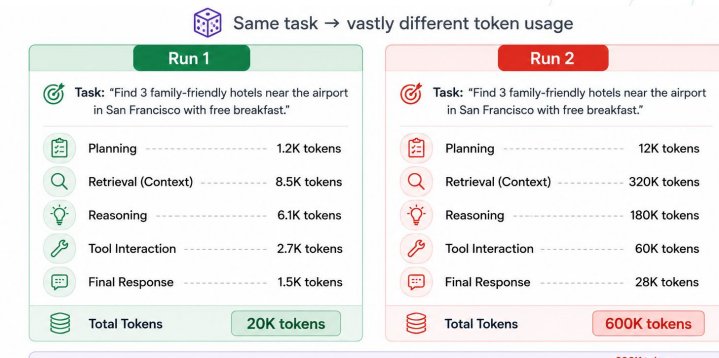
Reason:

- Different reasoning paths
- Tool retries
- Exploration loops
- Variable retrieval results

More tokens $\neq$ better accuracy

Agents may:

- Overthink
- Repeat failed reasoning
- Explore irrelevant branches



Even for same task and input, token consumption can vary by 10x to 30x or even more.





Managing Unpredictable Token Usage

Consumption Variability & Budget Control

Per-Trace Budgets

Organizations implement:

- Hard token ceilings
- Timeouts
- Maximum tool calls
- Loop detection safeguards

Cost observability is essential to prevent runaway agent behavior and massive API bills.



Thank You



WeCloudData

500-80 Bloor Street West, Toronto

ON

www.weclouddata.com

